

Saree Virtual Try-On API

A REST API that takes **only two photos** — the customer's photo and a photo of the saree — and returns a composite image of the customer "wearing" that saree.

No prompts. No generative AI (no diffusion/GAN image synthesis).

The output is produced entirely with classical computer vision:

1. **Pose detection** (MediaPipe PoseLandmarker) finds the customer's shoulders, hips and ankles in their photo.
2. **Body segmentation** (from the same model) identifies which pixels belong to the person.
3. The saree photo is **geometrically warped** (perspective transform) so its top band maps onto the torso and its bottom band maps onto the legs, following those body points.
4. The warped saree is **composited** onto the customer photo with a

feathered edge, masked to the segmented body area.

Because the saree is warped from the *actual uploaded photo*, the output always shows that exact saree's real colour, print and texture — nothing about the garment is invented or hallucinated.

Setup

```
pip install -r requirements.txt
python download_models.py    # one-time,
                             # needs internet — fetches the
                             # pose model
                             # file MediaPipe needs locally
python app.py                # starts the
                             # API on http://0.0.0.0:5000
```

`download_models.py` only downloads a generic, publicly published MediaPipe model file. It does not upload or transmit any customer or saree data anywhere — no photo ever leaves the server that runs this code.

API

GET /api/health

Returns `{"status": "ok"}`.

POST /api/tryon

multipart/form-data :

field	required	description
customer_photo	yes	jpg/png/webp, front-facing, full body preferred
saree_photo	yes	jpg/png/webp of the saree, laid flat / on a mannequin / clean product shot
drape_width_factor	no	float, default 1.18 — how far the drape extends past the body outline
feather	no	int, default 21 — edge-

field	required	description
		blend softness (px)

Response: `image/png` bytes of the composited result.

Error responses are JSON, e.g.:

```
{"error": "No person / body pose could be
detected in the customer photo."}
```

Example (curl)

```
curl -X POST
http://localhost:5000/api/tryon \
-F "customer_photo=@customer.jpg" \
-F "saree_photo=@saree.jpg" \
--output result.png
```

Example (Python)

```
import requests

with open("customer.jpg", "rb") as c,
open("saree.jpg", "rb") as s:
    r = requests.post(
        "http://localhost:5000/api/tryon",
        files={"customer_photo": c,
```

```
"saree_photo": s},
    )

with open("result.png", "wb") as out:
    out.write(r.content)
```

Files

File	Purpose
<code>app.py</code>	Flask API – request handling, validation, responses
<code>tryon_engine.py</code>	The CV pipeline: pose detection, warping, compositing
<code>download_models.py</code>	One-time fetch of the MediaPipe pose model file
<code>requirements.txt</code>	Python dependencies

Tips for best results

- **Customer photo:** front-facing, standing upright, most of the body visible, plain-ish background, good lighting.

- **Saree photo:** shot flat, on a mannequin, or as a clean draped product photo — avoid heavy background clutter around the fabric.
- If the drape looks too narrow/wide, tune `drape_width_factor` (e.g. 1.1 – 1.3).

Known limitations

- This is a **warp-and-blend approximation**, not a physical cloth simulation — it won't reproduce realistic pleats/folds the way a professionally shot photo would.
- Best with a single, mostly-upright, front-facing person; side poses, seated poses, or multiple people in frame reduce accuracy.
- Very baggy/loose customer clothing under the try-on can confuse the body segmentation mask at the edges.

Extending this

- Swap in a higher-fidelity approach (e.g. a proper cloth-mesh simulation on a fitted 3D body model) if you need

production-grade
realism later — the API surface (`POST /api/tryon`) wouldn't need
to change, only `tryon_engine.py` .

- Add caching / a job queue if you expect concurrent traffic — pose detection + warping is CPU-bound and takes roughly 0.5–2s per request on a typical server.